Gerenciamento de Tarefas, Tempo e Estados na ESP32

FreeRTOS • millis() • State Machines • Interrupções

ESP32

Agenda

01 Task Management na ESP32

02 O Problema do delay()

03 Controle de Tempo com millis()

04 Máquinas de Estado (State Machines)

05 Interrupções (GPIO & Timer)

06 Timer vs IO Interrupt

07 Código Prático (PlatformIO + ESP32)

Task Management na ESP32

FreeRTOS • Multitarefa • Concorrência vs Paralelismo

FreeRTOS na ESP32

A ESP32 executa o **FreeRTOS** nativamente, permitindo criar múltiplas tarefas independentes.

- Sistema operacional de tempo real (RTOS) embutido no ESP-IDF
- Dual-core: **PRO_CPU (Core 0)** + **APP_CPU (Core 1)**
- Escalonador preemptivo com prioridades (0–25)
- Cada task tem seu próprio `stack`, prioridade e estado
- Arduino `loop()` roda como uma task do FreeRTOS no Core 1
- API principal: `xTaskCreatePinnedToCore()`

Tasks vs Loop Tradicional

Loop Tradicional (Sequencial)

Ler Sensor
(bloqueado durante delay)

Atualizar Display
(espera anterior terminar)

Enviar WiFi
(tudo parado)

VS

FreeRTOS Tasks (Paralelo)

Task 1: Ler Sensor
Executa independente
Prioridade: Alta

Task 2: Atualizar Display
Executa independente
Prioridade: Média

Task 3: Enviar WiFi
Executa independente
Prioridade: Baixa

Concorrência vs Paralelismo

Concorrência

1 núcleo, múltiplas tarefas

- Tasks alternam rapidamente
- Ilusão de simultaneidade
- Time-slicing do scheduler
- Exemplo: ESP8266 (single-core)

A → B → A → B

Paralelismo

2 núcleos, execução simultânea real

- Tasks executam ao mesmo tempo
- Verdadeira simultaneidade
- Requer hardware multi-core
- **ESP32 = dual-core!**

A + B (simultâneo)

Quando Usar Multitarefa?

✓ Use multitarefa quando:

- Precisa ler sensores E comunicar WiFi simultaneamente
- Tarefas com períodos diferentes (100ms, 1s, 5s)
- Uma operação lenta não pode travar as outras

✗ Evite multitarefa quando:

- Projeto simples com uma única lógica sequencial
- RAM limitada (cada task consome ~2-8KB de stack)
- millis() + máquina de estados já resolve o problema

O Problema do delay()

Bloqueio de execução • Impacto em sistemas reativos

Por Que delay() é Prejudicial?

O `delay()` é uma função **bloqueante**: para TODO o processamento.

- ❌ CPU fica em busy-wait (não faz NADA)
- ❌ Sensores não são lidos durante o delay
- ❌ Comunicação serial/WiFi fica sem resposta
- ❌ Botões e entradas são ignorados
- ❌ Impossível gerenciar múltiplas tarefas com tempos diferentes
- ❌ Watchdog Timer pode resetar o ESP32

⚠ **delay(1000) = 1 segundo de CPU desperdiçada!**

Linha do Tempo: delay() vs Não-Bloqueante

COM delay()

Ler Sensor	**delay(1000) - BLOQUEADO**	Atualizar LED	**delay(500) - BLOQUEADO**

0ms 100ms 1100ms 1200ms

SEM delay()

Sensor	LED	WiFi	Sensor	LED	WiFi	Sensor	Serial	LED

0ms 100ms 200ms

CPU ociosa por longos períodos

↑ Bloqueante

CPU sempre produtiva, reativa

↑ Não-bloqueante

Código: delay() vs millis()

✗ Com delay()

```
void loop() {
  digitalWrite(LED, HIGH);
  delay(1000); // TRAVA!
  digitalWrite(LED, LOW);
  delay(1000); // TRAVA!
}
```

2s de CPU desperdiçada por ciclo

✓ Com millis()

```
unsigned long prev = 0;
void loop() {
  if(millis()-prev>=1000){
    prev = millis();
    toggle(LED); // livre!
  }
}
```

CPU livre entre execuções!

Controle de Tempo com millis()

Medir tempo sem bloquear • Execução periódica

Como millis() Funciona?

A função `millis()` retorna o tempo em ms desde o boot. Não bloqueia nada!

Padrão fundamental:

1. Guardar o `tempoAnterior` (unsigned long)
2. A cada loop, calcular: `millis() - tempoAnterior`
3. Se diferença ≥ intervalo desejado → executar ação
4. Atualizar `tempoAnterior = millis()`

⚠ Tipo: unsigned long (32 bits) → overflow em ~49 dias

A subtração `millis() - prev` funciona corretamente mesmo com overflow!

Padrão millis(): Fluxo de Execução

O loop() executa milhares de vezes por segundo

Ação só executa quando o intervalo é atingido

Aplicações Práticas de millis()

💡 Piscar LED sem bloquear

- Intervalo de 500ms, LED alterna HIGH/LOW
- Enquanto isso, outras tarefas continuam rodando

🌡 Leitura periódica de sensor

- DHT22 a cada 2s, BMP280 a cada 500ms
- Cada sensor com seu próprio `tempoAnterior`

⚙ Controle de processo industrial

- PID a cada 100ms + log a cada 5s + display a cada 1s
- Cada subsistema opera no seu tempo, sem interferir nos outros

Máquinas de Estado

State Machines • if/else • switch-case • Arquitetura escalável

Nível 1: if + millis()

Abordagem mais simples: variável de estado + **`if/else`** + millis()

```
bool ledState = false;
unsigned long prev = 0;
void loop() {
  if (millis() - prev >= 1000) {
    prev = millis();
    ledState = !ledState;
    digitalWrite(LED, ledState);
  }
}
```

✓ Simples, funciona para 2 estados (ON/OFF, IDLE/RUN)

Nível 2: switch-case

Código: switch-case State Machine

```
enum State { IDLE, READING, PROCESSING, SENDING };
State state = IDLE;
void loop() {
  switch(state) {
    case IDLE:
      if(millis()-prev>=2000) {
        state = READING; } break;
    case READING:
      valor = analogRead(SENSOR);
      state = PROCESSING; break;
    case PROCESSING:
      resultado = calcular(valor);
      state = SENDING; break;
    case SENDING:
      enviarWiFi(resultado);
      prev = millis(); state = IDLE; break;
  }
}
```

Nível 3: Arquitetura Escalável

Main Controller (loop)

↓

State Manager (switch-case)

↙ ↓ ↘

Módulo Sensor
sensorInit()
sensorRead()
sensorProcess()

Módulo Display
displayInit()
displayUpdate()
displayError()

Módulo Comunicação
wifiConnect()
mqttPublish()
httpPost()

✓ Cada módulo isolado
✓ Fácil de testar

✓ Reutilizável
✓ Escalável

✓ Fácil manutenção
✓ Debug simplificado

Comparação: Níveis de State Machine

Aspecto	Nível 1: if+millis	Nível 2: switch-case	Nível 3: Modular
Complexidade	Baixa	Média	Alta
Estados	2 (ON/OFF)	4-8 estados	Ilimitados
Estrutura	if/else simples	switch-case	Módulos + SM
Manutenção	★☆☆	★★☆	★★★
Escalabilidade	★☆☆	★★☆	★★★
Uso Típico	Blink LED Botão simples	Controle processo Menu LCD	IoT completo Robótica

Interrupções na ESP32

GPIO Interrupts • Timer Interrupts • Polling vs Interrupt

Polling vs Interrupção

Polling

"Verifica constantemente"

- ✕ CPU verifica a cada loop
- ✕ Gasta ciclos de CPU
- ✕ Pode perder eventos rápidos
- ✕ Latência variável

```
if(digitalRead(BTN))
```

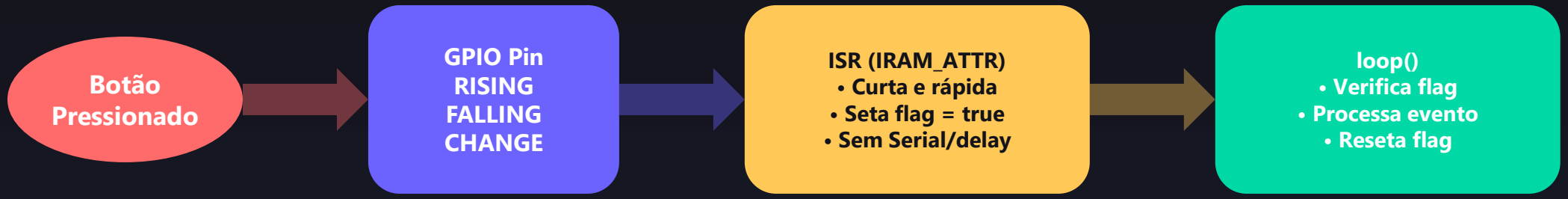
Interrupção

"Reage quando acontece"

- ✓ CPU livre até o evento
- ✓ Resposta imediata (~µs)
- ✓ Nunca perde eventos
- ✓ Latência baixa e constante

```
attachInterrupt()
```

Interrupção GPIO: Como Funciona



⚠ ISR deve ser IRAM_ATTR na ESP32
⚠ Use volatile para variáveis compartilhadas

Modos: RISING, FALLING, CHANGE,
LOW, HIGH (ESP32 suporta todos)

Tempo → | loop rodando... | ⚡ ISR (2μs) | loop continua... | processa evento | loop rodando...

Interrupção por Timer

Timer Interrupts geram interrupções em **intervalos precisos** de hardware.

Características da ESP32:

- 4 timers de hardware (64 bits cada)
- Clock base: `80 MHz` (prescaler configurável)
- Precisão de **microsegundos**
- Modo: auto-reload (periódico) ou one-shot

Uso típico:

- Amostragem precisa de ADC (ex: 10kHz)
- Geração de PWM por software
- Watchdog personalizado, timeout de comunicação

Timer vs IO Interrupt & Código Prático

Comparação técnica • PlatformIO + ESP32

Timer vs GPIO Interrupt

Característica	GPIO Interrupt	Timer Interrupt
Gatilho	Evento externo (botão, sensor)	Tempo decorrido (periódico)
Precisão	Depende do debounce ~1-50ms	Hardware clock ~1μs
Latência	Baixa (~2-5μs) Depende do evento	Muito baixa (~1-2μs) Previsível
Uso Típico	Botões, encoders Sensores digitais	Amostragem ADC PWM, Watchdog
Vantagem	Reativo a eventos Baixo consumo	Alta precisão temporal Determinístico
Limitação	Bounce mecânico Ruído elétrico	Nº limitado (4 timers) Consumo contínuo

Código: millis() & GPIO Interrupt

Controle com millis()

```
const int LED = 2;
unsigned long prev=0;
bool state=false;
void setup(){
  pinMode(LED,OUTPUT);
}
void loop(){
  if(millis()-prev>=500){
    prev=millis();
    state=!state;
    digitalWrite(LED,state);
  }
}
```

Interrupção GPIO

```
volatile bool btnFlag=false;
const int BTN=4, LED=2;
void IRAM_ATTR isr(){
  btnFlag=true;
}
void setup(){
  pinMode(BTN,INPUT_PULLUP);
  pinMode(LED,OUTPUT);
  attachInterrupt(BTN,isr,FALLING);
}
void loop(){
  if(btnFlag){
    btnFlag=false;
    digitalWrite(LED,!digitalRead(LED));
  }
}
```

Código: Timer Interrupt & State Machine

Timer Interrupt

```
hw_timer_t *timer=NULL;
volatile bool tmrFlag=false;
void IRAM_ATTR onTimer(){
  tmrFlag=true;
}
void setup(){
  // Timer 0, prescaler 80
  // = 1MHz (1µs/tick)
  timer=timerBegin(0,80,true);
  timerAttachInterrupt(
    timer,&onTimer,true);
  // 1s = 1000000µs
  timerAlarmWrite(
    timer,1000000,true);
  timerAlarmEnable(timer);
}
```

State Machine + millis()

```
enum S{IDLE,READ,SEND};
S st=IDLE;
unsigned long prev=0;
void loop(){
  switch(st){
  case IDLE:
    if(millis()-prev>=2000){
      st=READ;} break;
  case READ:
    val=analogRead(34);
    st=SEND; break;
  case SEND:
    Serial.println(val);
    prev=millis();
    st=IDLE; break;
  }
}
```

Resumo & Boas Práticas

Guia de Decisão Rápida:

- **delay()** → Só para debug/prototipagem rápida
- **millis()** → Controle de tempo não-bloqueante
- **State Machine** → Lógica complexa + múltiplos estados
- **GPIO Interrupt** → Resposta rápida a eventos externos
- **Timer Interrupt** → Amostragem precisa + watchdog

✅ **Boas Práticas:**

- ISR curta: setar flag, sair → processar no loop()
- Sempre `volatile` para variáveis entre ISR e loop()
- Combinar técnicas: millis() + SM + interrupt = sistema robusto